# Solution: Number of Flowers in Full Bloom

## Problem Statement

You are given a 2D integer array `flowers`, where **flowers[i] = [start$_i$, end$_i$]** means the **i$^{th}$** flower will be in `full bloom` from time **start$_i$** to **end$_i$** (inclusive). You are also given another 1D array `persons`, where `persons[i]` represents the time when **i$^{th}$** person will come to see the flowers.

Return an integer array answer of size n, where answer[i] is the number of flowers that are in full bloom at the **i$^{th}$** time.

### Examples

**Example 1**:

- **Input**: flowers = [[1,4],[2,2]], persons = [2,3]
- **Expected Output**: [2,1]
- **Explanation**: At time 2, both flowers are in full bloom. By time 3, the first flower is still in bloom, but the second one has wilted, leaving only one in bloom.

**Example 2**:

- **Input**: flowers = [[5,10],[15,25],[25,30]], persons = [5,10,15,20,25,30]
- **Expected Output**: [1,1,1,1,2,1]
- **Explanation**: At time 5 and 10, only the first flower is in bloom. At time 15 and 25, only the second flower is in bloom. At time 25, second and third flowers are in bloom, and at time 30 only third flower is in bloom.

**Example 3**:

- **Input**: flowers = [[1,50],[10,14],[6,10]], persons = [11,6,3]
- **Expected Output**: [2,2,1]
- **Explanation**: At time 11, the first and second flowers are in bloom. At time 6, the first and third flowers are in bloom. At time 3, only first flower is in bloom.

## Solution

To solve this problem, we employ a sweep line technique coupled with sorting. This approach is effective because it allows us to efficiently track changes in the state (number of flowers in bloom) over time. Firstly, we preprocess the flowers' bloom intervals by marking the start of a bloom with a positive indicator and the end with a negative indicator. This preprocessing step transforms the problem into one of tracking net changes in bloom count over time.

Sorting this modified list of events (bloom starts and ends) ensures that we encounter these events in chronological order. As we iterate through the sorted events, we adjust our count of flowers in bloom accordingly. For each person's query time, we perform a binary search on the sorted event list to find the number of flowers in bloom up to that point. This method is highly efficient for handling multiple queries, making it superior for scenarios with large datasets and numerous queries.

### Step-by-Step Algorithm

1. **Define Structures**:
   - Define an `event` struct with two fields: `time` (an integer) and `change` (also an integer, where +1 indicates a bloom start and -1 indicates a bloom end).
2. **Convert Flower Intervals to Events**:
   - For each flower interval `[start, end]` in the input list `flowers`, create two events:
     - A start event with `time = start` and `change = +1`.
     - An end event with `time = end + 1` and `change = −1`.
   - Add these events to a slice of `event` structs.
3. **Sort the Events**:
   - Sort the events first by `time` in ascending order. If two events have the same `time`, sort by `change` to ensure end events are processed before start events if they occur at the same time.
4. **Calculate Prefix Sum of Blooms**:
   - Initialize a variable `blooms` to keep track of the current number of flowers in bloom.
   - Iterate through the sorted events, updating `blooms` by adding the `change` value of each event.
   - Store the running total of `blooms` in a slice `prefixSum` at each step to keep track of the number of flowers in bloom after each event.
5. **Answer Each Query**:
   - For each query time in `persons`, perform a binary search on the events to find the last event that occurs at or before the query time.
   - Use the index from the binary search to find the corresponding value in `prefixSum`, which indicates the number of flowers in bloom at the query time.
   - If the binary search does not find a suitable event (returns -1), the result for that query is 0.
6. **Binary Search Implementation**:
   - Implement a binary search that searches through the sorted events to find the last event that occurs at or before a given query time.
   - Adjust the `left` and `right` pointers based on the mid-event's time compared to the query time, ensuring we find the exact or closest preceding event time.
7. **Return Results**:
   - For each query time, add the result (number of flowers in bloom) to a result slice and return this slice as the output of the `fullBloomFlowers` method.

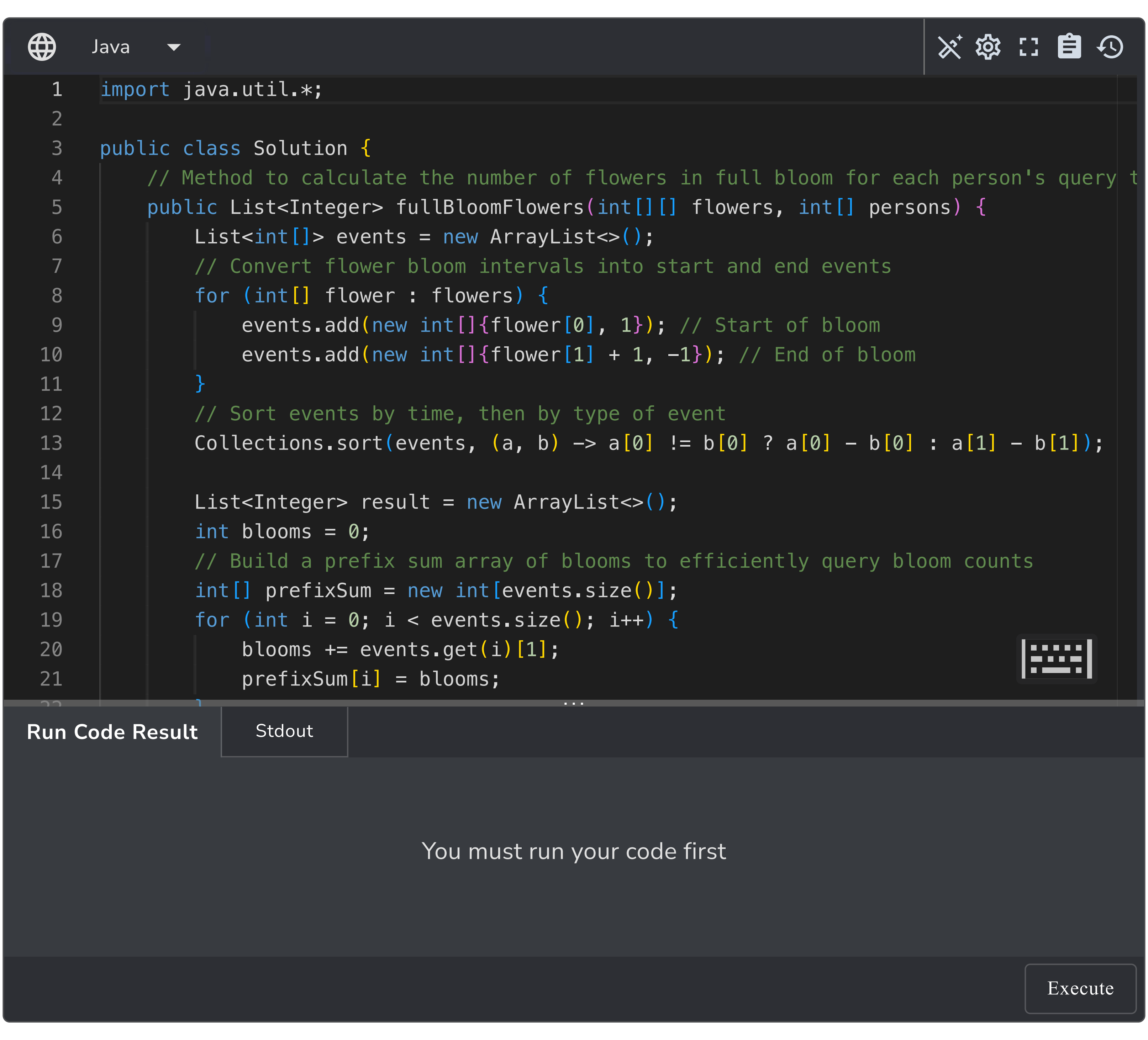
### Algorithm Walkthrough

**Input**: flowers = [[5,10],[15,25],[25,30]], persons = [5,10,15,20,25,30]

1. **Convert to Events**:
   - Events: [(5, +1), (11, -1), (15, +1), (26, -1), (25, +1), (31, -1)]
2. **Sort Events**:
   - Sorted Events: [(5, +1), (11, -1), (15, +1), (25, +1), (26, -1), (31, -1)]
3. **Calculate Prefix Sum**:
   - After event (5, +1): blooms = 1
   - After event (11, -1): blooms = 0
   - After event (15, +1): blooms = 1
   - After event (25, +1): blooms = 2
   - After event (26, -1): blooms = 1
   - After event (31, -1): blooms = 0
   - Prefix Sum: [1, 0, 1, 2, 1, 0]
4. **Answer Queries**:
   - For query time 5: Use binary search, found index 0. So, result = prefixSum[0] = 1.
   - For query time 10: Index 0. So, result = prefixSum[0] = 1.
   - For query time 15: Index 2. So, result = prefixSum[2] = 1.
   - For query time 20: Index 2. So, result = prefixSum[2] = 1.
   - For query time 25: Index 3. So, result = prefixSum[3] = 2.
   - For query time 30: Index 4. So, result = prefixSum[4] = 1.

**Output**: [1, 1, 1, 1, 2, 1]

## Code



```java
import java.util.*;

public class Solution {
    // Method to calculate the number of flowers in full bloom for each person's query t
    public List<Integer> fullBloomFlowers(int[][] flowers, int[] persons) {
        List<int[]> events = new ArrayList<>();
        // Convert flower bloom intervals into start and end events
        for (int[] flower : flowers) {
            events.add(new int[]{flower[0], 1}); // Start of bloom
            events.add(new int[]{flower[1] + 1, -1}); // End of bloom
        }
        // Sort events by time, then by type of event
        Collections.sort(events, (a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] - b[1]);

        List<Integer> result = new ArrayList<>();
        int blooms = 0;
        // Build a prefix sum array of blooms to efficiently query bloom counts
        int[] prefixSum = new int[events.size()];
        for (int i = 0; i < events.size(); i++) {
            blooms += events.get(i)[1];
            prefixSum[i] = blooms;
```

You must run your code first

## Complexity Analysis

### Time Complexity

1. **Creating Events**: For `n` flowers, we create `2n` events (start and end for each flower). This step takes $O(n)$ time.
2. **Sorting Events**: Sorting `2n` events takes $O(2nlog(2n)) = O(nlogn)$ time.
3. **Prefix Sum Calculation**: Calculating the prefix sum for bloom counts across all events takes $O(2n) = O(n)$ time, as we iterate through each event once.
4. **Answering Queries**: For each of `m` persons, we perform a binary search among the events to find the bloom count. Each binary search takes $O(logn)$ time, leading to a total of $O(mlogn)$ for this step.

Overall, the time complexity is $O(nlogn + mlogn)$, where `n` is the number of flowers and `m` is the number of persons.

### Space Complexity

1. **Event Storage**: Storing `2n` events requires $O(2n) = O(n)$ space.
2. **Prefix Sum Array**: The prefix sum array also requires $O(2n) = O(n)$space.
3. **Output Array**: The output array for `m` queries requires $O(m)$ space.

The space complexity is therefore $O(n + m)$, accounting for the storage of events, prefix sums, and the output array.

← Previous
Number Of Flowers In Full Bloom (Hard)

Next →
Median Of Two Sorted Arrays (Hard)

☐ Mark as Completed